

3. Documentación Técnica

Despliegue Realizado

Despliegue Local (Actual)

El proyecto se encuentra actualmente en ejecución en un entorno local:

Componente	Detalle
Servidor	Django Development Server
URL	http://localhost:8000
Puerto	8000
Base de Datos	SQLite (backend/db.sqlite3)
Sistema Operativo	Windows
Python	3.9+
Estado	Operativo (HTTP 200)

Despliegue con Docker

El proyecto incluye docker-compose.yml para despliegue contenerizado:

```
services:
  db:
    # PostgreSQL 15 Alpine
    image: postgres:15-alpine
    ports: ["5432:5432"]
    environment:
      POSTGRES_DB: healthcare_db
      POSTGRES_USER: healthcare_user
      POSTGRES_PASSWORD: healthcare_password
  web:
    # Django + Gunicorn
    build: ./backend
    command: python manage.py runserver 0.0.0.0:8000
    ports: ["8000:8000"]
    depends_on: [db]
    env_file: .env
```

Para ejecutar con Docker:

```
docker-compose up --build
```

Despliegue en la Nube (Guía para Producción)

Para desplegar en plataformas como Render, Railway o Heroku:

Preparar settings.py para producción:

```
python DEBUG = False ALLOWED_HOSTS = ['tu-app.onrender.com']
```

Configurar PostgreSQL como base de datos:

```
python # Usar DATABASE_URL de variable de entorno import dj_database_url
DATABASES['default'] = dj_database_url.config()
```

Archivo de inicio (build.sh):

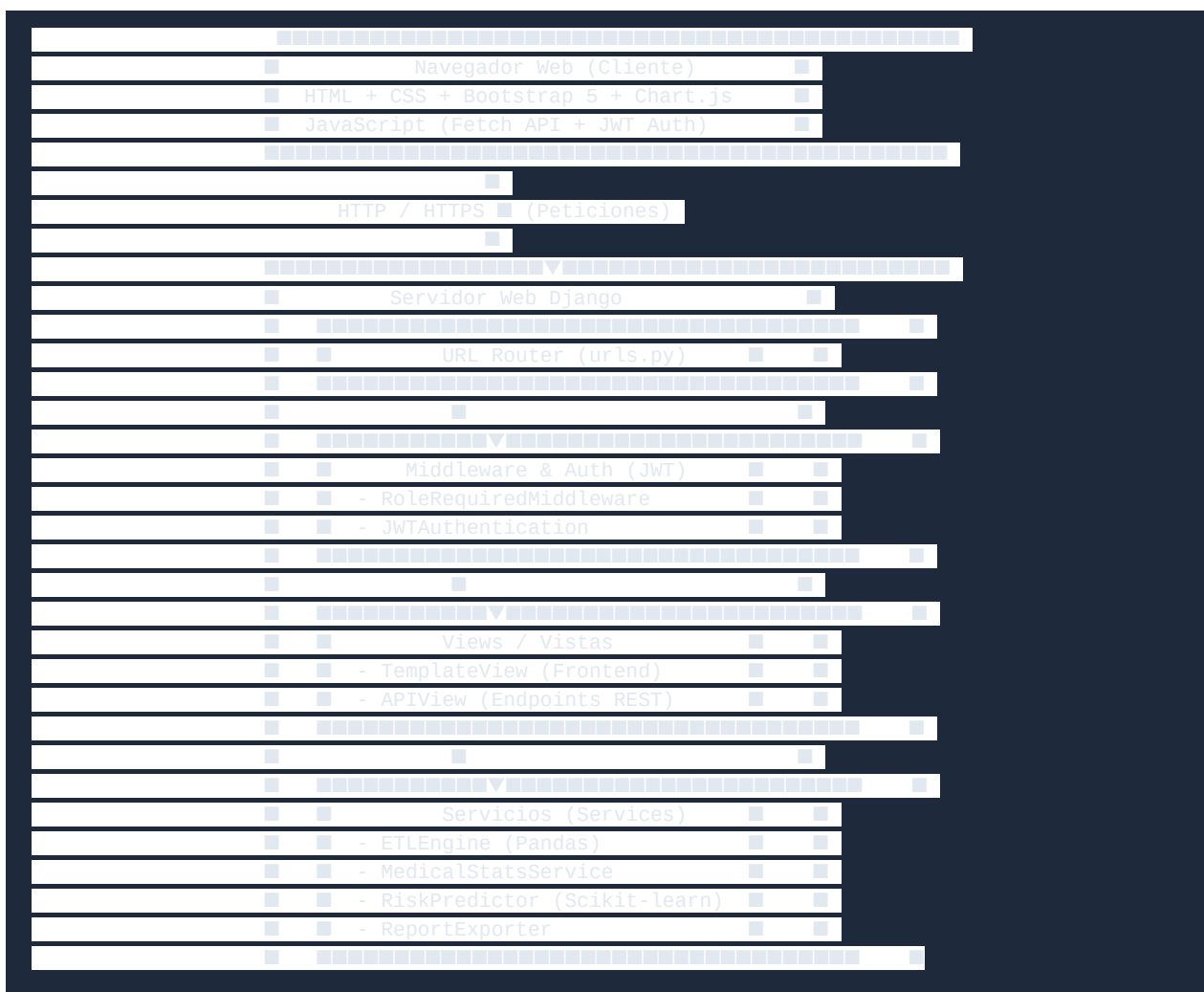
```
bash pip install -r requirements.txt python manage.py collectstatic --no-input python
manage.py migrate python manage.py seed gunicorn config.wsgi:application --bind
0.0.0.0:$PORT
```

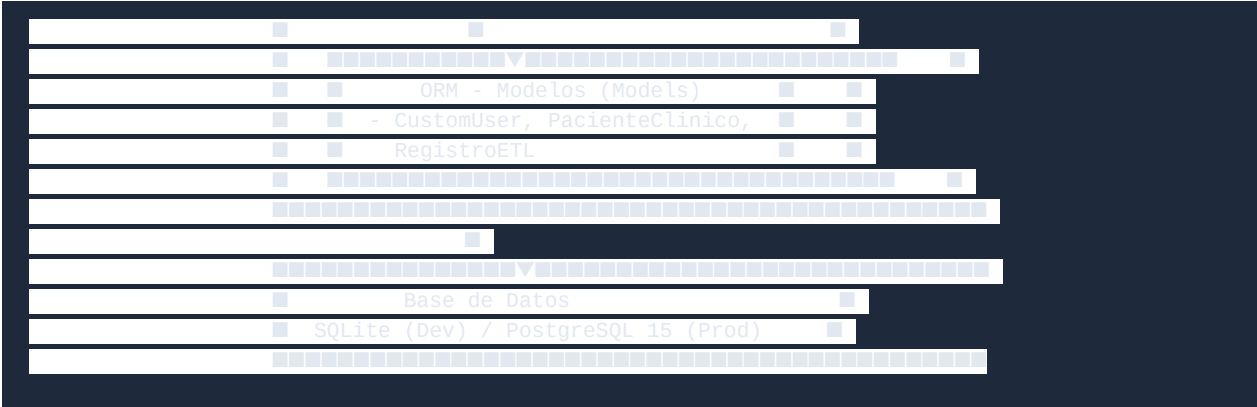
Variables de Entorno requeridas:

- SECRET_KEY - Clave secreta de Django
- DATABASE_URL - URL de conexión PostgreSQL
- ALLOWED_HOSTS - Hosts permitidos
- DEBUG=False

Arquitectura del Sistema

Patrón: Cliente-Servidor con Django MVT (Model-View-Template)





Componentes del Sistema

Componente	Tecnología	Función
Backend	Django 4.2 + DRF 3.17	Lógica de negocio, API REST, autenticación
Base de Datos	SQLite / PostgreSQL	Almacenamiento persistente de datos
Frontend	Bootstrap 5 + Chart.js	Interfaz de usuario responsiva
Autenticación	JWT (SimpleJWT)	Seguridad basada en tokens
ETL	Pandas + OpenPyXL	Extracción, transformación y carga de datos
Machine Learning	Scikit-learn (Random Forest)	Predicción de riesgo clínico
Reportes	ReportLab + OpenPyXL	Generación de PDF, Excel y CSV
Documentación API	drf-spectacular (Swagger)	Documentación interactiva de endpoints

Flujo de Autenticación

1. Usuario ingresa credenciales en `/login/`
2. Frontend envía POST a `/api/auth/login/`
3. Backend valida y retorna tokens JWT (access + refresh)
4. Tokens almacenados en `localStorage`
5. Cada petición API incluye `Authorization: Bearer <token>`
6. Middleware y DRF validan el token en cada request
7. Si el token expira, se usa `/api/auth/refresh/` para renovarlo
8. Roles de usuario controlan la visibilidad de secciones y permisos

APIs del Sistema

Endpoints Públicos

Método	Endpoint	Descripción	Permiso
POST	/api/auth/login/	Inicio de sesión (obtener JWT)	Público
POST	/api/auth/refresh/	Renovar token JWT	Público

Endpoints Autenticados

Módulo Autenticación

Método	Endpoint	Descripción	Permiso
GET	/api/auth/me/	Información del usuario actual	Autenticado
GET	/api/auth/usuarios/	Listar todos los usuarios	Admin
POST	/api/auth/usuarios/	Crear nuevo usuario	Admin
PUT	/api/auth/usuarios/{id}/	Actualizar usuario	Admin
DELETE	/api/auth/usuarios/{id}/	Eliminar usuario	Admin

Módulo ETL

Método	Endpoint	Descripción	Permiso
POST	/api/etl/run/	Ejecutar ETL sobre dataset predeterminado	Admin/Analista
POST	/api/etl/upload/	Subir y procesar archivo CSV/Excel	Admin/Analista
GET	/api/etl/historial/	Historial de ejecuciones ETL	Admin/Analista
GET	/api/etl/pacientes/	Lista paginada de pacientes (filtro por sexo/riesgo)	Autenticado

Módulo Analytics

Método	Endpoint	Descripción	Permiso
GET	/api/analytics/kpis/	KPIs principales (total, críticos, hipertensos, diabéticos)	Autenticado

GET	/api/analytics/estadisticas/	Estadísticas descriptivas (media, mediana, moda, std)	Autenticado
GET	/api/analytics/segmentacion/	Distribuciones por sexo, edad, riesgo, diagnóstico, IMC	Autenticado

Módulo Machine Learning

Método	Endpoint	Descripción	Permiso
POST	/api/ml/entrenar/	Entrenar modelo Random Forest	Admin/Analista
POST	/api/ml/predecir/	Predecir riesgo de un paciente	Autenticado

Módulo Reportes

Método	Endpoint	Descripción	Permiso
GET	/api/reportes/exportar/	Exportar datos (csv, excel, pdf)	Autenticado

Frontend (Vistas HTML)

Ruta	Vista	Descripción
/	Dashboard	Panel analítico con KPIs y gráficas
/login/	Login	Página de inicio de sesión
/etl/	ETL	Panel de control del pipeline ETL
/pacientes/	Pacientes	Directorio maestro de pacientes
/ml/	ML	Panel de Machine Learning
/reportes/	Reportes	Exportación y estadísticas
/usuarios/	Usuarios	Gestión de usuarios (Admin)
/admin/	Admin Django	Panel de administración de Django
/api/docs/	Swagger UI	Documentación interactiva de la API

Ejemplos de Uso de la API

1. Login:

```
curl -X POST http://localhost:8000/api/auth/login/ \
-H "Content-type: application/json" \
-d '{"username": "admin", "password": "Admin1234"}'
```

2. Obtener KPIs:

```
curl http://localhost:8000/api/analytics/kpis/ \
-H "Authorization: Bearer <access_token>"
```

3. Ejecutar ETL:

```
curl -X POST http://localhost:8000/api/etl/run/ \
-H "Authorization: Bearer <access_token>"
```

4. Predecir riesgo:

```
curl -X POST http://localhost:8000/api/ml/predecir/ \
-H "Authorization: Bearer <access_token>" \
-H "Content-Type: application/json" \
-d '{
  "edad": 65, "IMC": 32.5, "presion_sistolica": 160,
  "presion_diastolica": 95, "glucosa": 180, "colesterol": 250,
  "frecuencia_cardiaca": 85, "fumador": 1
}'
```